

PREFECT PIPELINES,
ENGINEERED

Prefect: Long-running flows that never lose their place.

Why is failure recovery in your pipelines still a manual task?
Automate resilience, not...

+117k
downloads

**<50
MB**
RAM, total

SQLite
WAL state

THE CONTEXT

Where it hurts today

Long runs lose state

Workers + server + fleet

Observability gets costly

THE SHIFT

What you gain



Code-first

Pipelines are Python — versionable, testable.



Resume, don't restart

Every step checkpoints to the WAL.

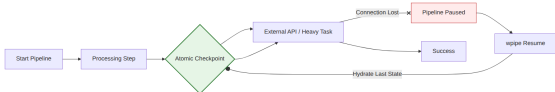


Featherweight

<50 MB. No broker, no daemon.

THE ARCHITECTURE

One pipeline, fully instrumented



State, retries and audit — inside a single run.

SIDE BY SIDE

wpipe vs. the status quo

Feature	SaaS / Cloud Approach	wpipe (Sovereign) Approach
State Persistence	Metadata in an external DB	Atomic Data Context in SQLite WAL
Recovery	Task re-dispatch from server	Local-first state continuity
Complexity	High (Agent/flow management)	Minimal (`@step` decorator)
Reliability	Subject to orchestrator connectivity	Immune to external network failures

SEE IT IN ACTION

Six lines. That's it.

```
from wpipe import Pipeline, step

@step(name="resume_me",
checkpoint=True)
def resume_me(data):
    return {"state":
hydrate(data["cursor"])}

```

CHECKPOINTS · RETRIES · TIMEOUT ·
PARALLEL · TRACKER

Try wpipe in your next pipeline ✓

Install the library, wrap your step
— checkpoints and docs appear
by themselves.

```
pip install wpipe
```

wpipe

Python-native orchestration